V12 地标与导航系统研究

Executive Summary

`godot_citys` 现有资产已经具备 shared `road_graph`、`vehicle lane graph`、`intersection turn contract`、minimap 投影和基础 route overlay，但离 GTA5 式“地标检索 -> 选点设目的地 -> 路线高亮 -> 偏航重算 -> HUD 提示 -> 全屏地图 -> 快速消费”还差一层统一的位置语义与查询 contract。结合 GTA5 可观察产品语义、现代导航 API 的通行建模，以及仓库当前的 world generation / cache / lane graph 实情，`v12` 应当被定义成一个多里程碑大版本：先冻结地址和地标元数据，再交付 lane-based routing，最后让 minimap、全屏地图、Task 图钉、瞬移和自动驾驶都消费同一份 route contract，而不是各自重复猜测。[1][2][3][4][6][7][8][9]

本地盘点也说明这不是一个小 demo：当前默认 seed `424242` 下，世界里已经有 `31,860` 条道路边、`10,621` 个交叉点、`7,640` 个现有唯一道路显示名、`89,679` 条 vehicle lane、`300,304` 个 block 和 `1,201,216` 个 parcel。`v12` 的命名、编号、缓存与 UI 设计必须按“大世界地址系统”来做，而不是按几十个 POI 的手工表来做。

Key Findings

- **GTA5 式导航的核心是统一的“目的地语义链”，不是单独一条 minimap 线**：官方 companion app 与社区文档共同表明，GTA 的地图体验强调全图浏览、目的地选择、地图标记、GPS 路线和高频入口，而不是把 map/minimap/HUD/Quick GPS 当成彼此独立的功能。[1][2][3][4]
- **地标系统和导航系统必须共用同一个 `Place Index`**：现代导航系统把 `address / place / waypoint / snapped road point / route steps` 当作同一条数据链；如果查询系统和 routing 系统不是同源，后续 HUD、Task 图钉、瞬移和自动驾驶一定会漂移。[6][7][8][9]
- **地址应该是结构化字段，不该只是 display string**：业界通用 geocoding 与邮政标准都把 `house_number`、`road`、`place`、`region` 视为结构化字段；这直接支持 `godot_citys` 用“block-based 门牌 + canonical street cluster”做稳定地址，而不是把完整地址一次性拼成一串随机文本。[6][10]
- **自动驾驶不应和 routing 混写**：route planning 负责“去哪、怎么走”，HUD 负责“怎么提示”，fast travel 负责“如何瞬时消费”，auto-drive 负责“如何沿 route 控车”；把这几层混到一起会让后续维护和性能证明都失真。[8][9][12][13]
- **现有仓库最值得复用的路由底盘不是 `CityMacroRouteGraph`，而是 `vehicle lane graph + intersection turn contract`**：当前 `CityMacroRouteGraph` 仍是 chunk-level

Manhattan route，而 `vehicle_query` 已经构建出 `89,679` 条 lane 和 `10,621` 个 turn contract，这才是 v12 正式 routing 的正确上游。

Detailed Analysis

GTA5 的产品语义：地图不是 UI 附件，而是位置语义入口

从公开可观察的 GTA5 语义看，地图系统至少承担五件事：一是让玩家浏览整座城市；二是设置 waypoint / 目的地；三是把目的地投影到雷达或 minimap；四是给驾驶态玩家持续的 GPS 导航反馈；五是通过快捷入口把常见类别目标快速设为当前导航目标。[1][2][3][4]

这意味着，`v12` 里的 `M` 全屏地图不能只是“把当前 minimap 放大”。它必须具备完整的世界浏览、目的地选择和泛型 pin overlay 能力。用户刚补充的“按下 `M` 打开全地图、世界暂停、可直接选点、未来渲染 Task 图钉”本质上不是附加项，而是 GTA 语义的核心部分。[1][3][4]

这里需要明确一个推断：Rockstar 的公开文档不会告诉我们内部 route solver 如何实现，但从 GPS、Quick GPS、地图标记、任务图钉、驾驶态指引这些稳定可观察行为来看，可以合理推断它们消费的是同一层位置解析和 route state，而不是每个 UI 各自临时做一套逻辑。这一点与现代导航 SDK 的架构也一致。[2][4][7][8][9]

地标、道路、地址：应先建 canonical `Place Index`

Kevin Lynch 的经典城市意象框架把城市可读性拆成 `paths / edges / districts / nodes / landmarks` 五类对象。[5] 对 `godot_citys` 来说，这个框架正好对应现有资产：

- `paths`：shared `road_graph` 与 `vehicle lane graph`
- `nodes`：`intersection turn contract`
- `districts`：当前 `district_graph`
- `landmarks`：v12 新增的可命名建筑、地标和任务 pin
- `edges`：世界边界、桥、高架、河道或大块地形边缘，后续可增补

因此，`v12` 不该从“给每条 edge 起个名字”开始，而应从“定义什么是 place”开始。推荐的 canonical place 类型至少包括：

- `road_cluster`
- `intersection`
- `addressable_building`
- `landmark`
- `task_pin`

- `arbitrary_map_point`

每个 place 都应至少有：

- `place_id`
- `place_type`
- `display_name`
- `normalized_name`
- `world_anchor`
- `routable_anchor`
- `district_id`
- `search_tokens`
- `source_version`

其中 `addressable_building` 不能直接依赖当前 chunk-local building mesh。当前 building 是 `CityChunkProfileBuilder` 在 chunk 级按 deterministic 规则现算出来的，并没有全局稳定索引；真正适合做地址主键的是 `block_layout` 及其 parcel 层。也就是说，`v12` 应把“建筑地址”定义在 `parcel / frontage slot` 这一层，再由运行时建筑去绑定它，而不是反过来。

路名数量与编号规则：按 street cluster 定量，不按 raw segment 定量

本地盘点显示当前世界共有 `31,860` 条道路边，但只有 `7,640` 个唯一显示名。这已经证明“路名数量”天然不应等于 raw segment 数量。进一步看当前命名内容，里面还混有 `Reference 00000` 和 `district_xx_yy Connector` 这类明显占位的技术名，因此 v12 必须引入正式的 `street cluster` 概念。

推荐做法：

- 以 lane/road continuity 为准，把连续可驾驶走廊聚成 canonical `street_cluster`
- road name 数量按 `street_cluster_count` 决定，不按 edge 数决定
- 目标带宽先按 `6,000 +/- 1,000` 条 canonical street cluster 设计
- AI 生成的道路名候选池至少按最终 street cluster 数的 `1.8x` 准备，建议首轮落在 `11,000` 到 `13,000` 个 root candidates

地址编号推荐直接采用美式 block-based 语法，而不是 AI 随机编：

- 每个 street cluster 沿 canonical increasing direction 切成 block face
- 每过一个交叉口，门牌百位进一段
- 左右两侧奇偶分离
- block 内 frontage slot 用 `02, 04, 06 ...` 或 `01, 03, 05 ...` 编号

- 展示格式默认为 `1120 Jefferson Road`

这套规则和 USPS Publication 28 的结构化地址思想兼容，也便于后续做地址搜索、HUD 文案和任务目标解析。[10]

Routing：应切到 lane-based route contract，而不是继续 chunk Manhattan

当前仓库虽然已经有 `plan_macro_route()` 和 minimap route overlay，但 `CityMacroRouteGraph` 本质上仍是 chunk Manhattan path。这足够支撑 v3 的“大致有条线”，但不足以支撑你现在要的：

- 路名反查坐标
- HUD 左转/右转提示
- 偏航后 route refresh
- 自动驾驶
- 与 Task 图钉同源的 route target resolution

`vehicle_query` 已经构建好真正可复用的上游：

- `89,679` 条 vehicle lane
- `10,621` 个 `intersection turn contract`
- 每条 lane 绑定 `road_id / road_class / template_id`

因此 v12 的 route solver 应直接建立在 `vehicle lane graph` 之上，并返回富语义 route object，而不是只返回一条 polyline。推荐最低 contract：

- `route_id`
- `origin_place`
- `destination_place`
- `snapped_origin`
- `snapped_destination`
- `lane_path`
- `polyline`
- `legs`
- `steps`
- `maneuvers`
- `distance_m`
- `estimated_time_s`
- `reroute_generation`

其中 `steps/maneuvers` 至少要包含：

- `turn_type`
- `distance_to_next_m`
- `road_name_from`
- `road_name_to`
- `world_anchor`
- `instruction_short`

这套设计与现代导航 API 的普遍做法一致，后续无论是 minimap 高亮、HUD 提示、Task 追踪、瞬移还是自动驾驶，都不需要再各自重新猜路。[7][8][9][13]

全屏地图、Task 图钉、暂停世界：这是导航消费层，不是 route solver 本体

你补充的 `M` 全屏地图和“世界静止”要求非常关键。单机 Godot 项目里，这一层推荐被定义成 `Map Screen Consumer`：

- 输入：`Place Index`、`Route Query`、`Pin Registry`
- 行为：打开全图、暂停 3D 世界、选择任意目标、设置当前 destination、显示图钉与图例
- 输出：`selected_target` 或 `active_destination`

这里最好直接约束成一套 pin contract，而不是把 Task 系统写死：

- `pin_id`
- `pin_type`
- `world_position`
- `title`
- `subtitle`
- `priority`
- `icon_id`
- `is_selectable`
- `route_target_override`

这样 v12 可以先把地图 pin 系统做成泛型 overlay；等未来 Task 系统落地后，只要往 pin registry 注册新的数据源即可，不用重写 map UI。

Fast Travel 与 Auto-Drive：同源消费，不共用控制逻辑

Fast travel 与 auto-drive 都应该消费同一份 `resolved destination + route contract`，但它们不该共用控制逻辑。

推荐拆法：

- `fast travel`
  - 输入：`resolved_target`
  - 输出：安全落点 teleport
  - 不关心 route follow 控制
- `auto-drive`
  - 输入：`route_result`
  - 输出：玩家当前 hijacked vehicle 的 steering / throttle / braking command
  - 不重新求路，只在偏航时请求 reroute

`Stanley` 论文与大世界 pathfinding 工程经验都支持这个分层：route solver 和 vehicle controller 是两个不同问题，混在一起只会让调试与性能证明都变差。[12][13]

对 `godot_citys` 的直接落地建议

推荐 v12 按以下路径推进：

1. 在 `CityWorldGenerator` 中，于 `road_graph + block_layout + vehicle_query` 稳定后新增 `place_index / place_query / route_target_index`
2. place 数据单独 cache 到 `user://cache/world/place_index_*.bin`，不要塞进 chunk render 现场临时生成
3. 用 `CityBlockLayout` 的 parcel 层扩展成 frontage / address slot，再让 chunk building 绑定它
4. 正式 route planner 复用 `vehicle lane graph + intersection turn contracts`
5. `CityMinimapProjector` 升级为 route result consumer，而不是 polyline-only painter
6. 新增全屏地图 UI、pin registry、pause-safe destination selection
7. fast travel 与 auto-drive 各自消费同一份 route result

Areas of Consensus

- 路径求解不应只返回“有一条线”，而应返回可供多消费者复用的富语义 route object。[7][8][9]
- 地址应是结构化字段，门牌和道路名应可被独立解析与搜索。[6][10]
- 地图、minimap、HUD、Quick GPS/快捷目标、Task pin 最终都应消费同一份位置语义，而不是多套不一致的临时逻辑。[1][2][3][4]

- route planning 与 vehicle control 应分层，自动驾驶应视为 route consumer，而不是 route solver 内部的一部分。[12][13]

Areas of Debate

- **命名风格**：更接近洛圣都的美式英语街名，还是中英混合。当前研究建议默认采用“完全虚构但沿用美式地址语法”，以避免直接照搬 GTA/现实城市专名。
- **地标覆盖率**：每个 parcel 都是否需要独立可搜索 building name。当前更合理的选择是“每个 address slot 可解析，但只有少量 landmark 有 proper name”。
- **自动驾驶强度**：v12 是只做玩家车的基础自动跟路线，还是进一步做复杂交通礼让。研究结论倾向前者，后者应另立后续版本。

Sources

[1] Rockstar Games, *Grand Theft Auto V: The Manual* app listing, Google Play. 官方 companion app 入口，说明 GTA V 地图/任务/位置浏览属于正式产品层能力。
https://play.google.com/store/apps/details?id=com.rockstargames.gtavmanual

[2] GTA Wiki, *GPS*. 社区整理的 GTA 导航可观察行为，适合产品语义研究，不适合作为内部实现证据。
https://gta.fandom.com/wiki/GPS

[3] GTA Wiki, *Maps*. 社区整理的 GTA 地图、标记与地图界面行为。https://gta.fandom.com/wiki/Maps

[4] GTA Wiki, *Interaction Menu*. 包含 Quick GPS 等高频目标入口，是 GTA 式“类别直达”产品语义的重要证据。https://gta.fandom.com/wiki/Interaction_Menu

[5] Wikipedia, *The Image of the City*. Kevin Lynch 城市可读性五要素的二级总结，适合作为地标/道路/区域分层设计的概念引导。https://en.wikipedia.org/wiki/The_Image_of_the_City

[6] Mapbox Docs, *Geocoding and Search APIs*. 官方 place/address 解析文档，说明现代导航系统把 address 与 place 视为结构化检索对象。https://docs.mapbox.com/api/search/

[7] Mapbox Docs, *Directions API*. 官方 route contract 文档，展示 `waypoints / legs / steps / intersections` 等标准结果结构。
https://docs.mapbox.com/api/navigation/directions/

[8] Mapbox Docs, *Maneuver API / Maneuver View*. 官方 HUD turn-by-turn consumer 组件文档，说明 maneuver 应作为 route result 的正式输出，而非 UI 现场猜测。
https://docs.mapbox.com/android/navigation/guides/ui-components/maneuver/

[9] Mapbox Docs, *Rerouting*. 官方偏航重算指南，说明 reroute 是 route lifecycle 的一部分，而不是另一套临时流程。https://docs.mapbox.com/ios/navigation/v2/guides/turn-by-turn-navigation/rerouting/

[10] United States Postal Service, *Publication 28: Postal Addressing Standards*. 官方门牌和街道缩写规范，可作为 block-based 地址语法的结构参考。https://pe.usps.com/text/pub28/

[11] Steve Rabin, *Game AI Pro, Chapter 17: Pathfinding Architecture Optimizations*. 专业游戏工程实践资料，说明大世界 pathfinding 需要做层级化与接口解耦。https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter17_Pathfinding_Architecture_Optimizations.pdf

[12] Sebastian Thrun et al., *Stanley: The robot that won the DARPA Grand Challenge*, Journal of Field Robotics. 技术论文，支持“route planning 与 vehicle control 分层”的工程原则。https://robots.stanford.edu/papers/thrun.stanley05.pdf

[13] Microsoft Research, *Alternative Routes in Road Networks*. 路由结果不应锁死成唯一折线，后续扩展替代路线和 richer consumer 需要更宽的 route contract。https://www.microsoft.com/en-us/research/wp-content/uploads/2013/01/jea_alt_final.pdf

Gaps and Further Research

- 需要在本地正式跑出 `street continuity clustering`，把“预计 `6,000 +/- 1,000` 条 canonical street cluster”收敛成真实数字。
- 需要决定命名风格是否保持“英语美式地址语法”，或改为中英混合。
- 需要在实现前明确“自动驾驶”的 v12 口径：是基础 player-only route follow，还是包含交通礼让和复杂避障。
- 需要在实现前把 `parcel -> frontage slot -> addressable building` 的数据模型写硬，否则地址系统会再次退回 runtime 临时拼装。